Guia de Engenharia de IA — AgendaVet

Do Fundamento ao Sistema Autônomo

Para quem é este guia? Este documento assume que você já sabe programar (JavaScript/TypeScript) mas quer entender profundamente como a IA funciona por dentro, como ela está integrada no AgendaVet hoje, e como evoluir o sistema de forma inteligente. Não é para iniciante absoluto em programação, mas explica os conceitos de IA do zero.

Índice

1. Como um Modelo de Linguagem Funciona de Verdade
2. O que Acontece Quando Você Manda uma Mensagem
3. Tokens — A Unidade de Trabalho da IA
4. Context Window — A Memória de Curto Prazo
5. System Prompt — Como Programar o Comportamento da IA
6. Tool Calling (Function Calling) — Como a IA Acessa Dados Reais
7. Streaming — Por que a IA "Digita" em Tempo Real
8. RAG — Retrieval Augmented Generation
9. Embeddings — Como a IA Entende Semântica
10. A Arquitetura do VetCopilot no AgendaVet
11. Como Escrever Prompts Poderosos

12. Gerenciamento de Custos e Tokens
13. O que Pode Ser Implementado — Mapa Completo
14. Próximos Passos Concretos com Código
15. Riscos, Limitações e Ética
16. Glossário Técnico

---

1. Como um Modelo de Linguagem Funciona de Verdade

1.1 A Ideia Central

Um LLM (Large Language Model) como Claude ou GPT **não pensa**. Ele **prediz a próxima palavra** (tecnicamente, o próximo token) com base em tudo que veio antes. É uma função matemática gigantesca que aprendeu padrões de bilhões de textos.

```
Entrada: "O gato está com febre e"
IA pensa: qual palavra vem depois mais provavelmente?
Saída:   "vômito"  (probabilidade: 12%)
         "letargia" (probabilidade: 9%)
         "diarreia" (probabilidade: 8%)
         ...
IA escolhe "vômito" → agora prediz a próxima → e assim por diante
```

1.2 A Arquitetura Transformer

O mecanismo que torna isso possível se chama **Transformer**, criado em 2017. O segredo dele é a **Atenção (Attention)**.

```
Frase: "O paciente Bolinha (Golden Retriever, 5 anos) chegou com tosse seca"

O mecanismo de atenção faz:
  "tosse" → presta atenção em "seca" (modifica o diagnóstico)
  "tosse" → presta atenção em "Retriever" (raça importa para algumas doenças)
  "tosse" → presta atenção em "5 anos" (idade importa para prognóstico)
```

```
Resultado: o modelo entende que "tosse seca em Golden Retriever de 5 anos"
           é diferente de "tosse produtiva em filhote de 2 meses"
```

Por que isso importa para você: Quanto mais contexto relevante você der ao modelo no prompt, mais precisa é a atenção que ele aplica. Um prompt genérico produz resposta genérica.

1.3 Treinamento vs Inferência

FASE	O QUE É	QUANDO ACONTECE	VOCÊ CONTROLA?
Pré-treinamento	O modelo aprende com trilhões de tokens da internet	Uma vez, pela Anthropic/OpenAI	Não
Fine-tuning	Ajuste para uma tarefa específica	Após pré-treino	Sim, mas caro
RLHF	Humanos ensinam o que é "boa" resposta	Durante treinamento	Não
Inferência	O modelo responde sua pergunta	Toda vez que você chama a API	Sim — é aqui que você trabalha

Conclusão prática: Como desenvolvedor, você trabalha exclusivamente na fase de **inferência** — a cada chamada de API. Seu poder está em: o que você coloca no prompt, quais ferramentas você dá ao modelo, e como você processa a saída.

---

2. O que Acontece Quando Você Manda uma Mensagem

Veja o fluxo completo de uma pergunta no VetCopilot do AgendaVet:

```
Veterinário digita: "Quais vacinas o Bolinha está atrasado?"
          |
```

```
          ▼
[FRONTEND - React]
  → Captura a mensagem
  → Adiciona ao histórico de chat (array de mensagens)
  → Faz POST para /api/vet-copilot
          |
          ▼
[API ROUTE - Next.js]
  → Verifica autenticação (JWT do Supabase)
  → Carrega contexto do pet (nome, idade, espécie, histórico)
  → Monta o array de mensagens completo:
      [
        { role: "system", content: "Você é o VetCopilot..." },
        { role: "user",   content: "Pet: Bolinha, 5 anos..." },  ← contexto
        { role: "user",   content: "Quais vacinas estão atrasadas?" }
      ]
  → Chama a API da Claude (via OpenRouter)
          |
          ▼
[MODELO DE IA - Claude Sonnet]
  → Recebe todos os tokens de uma vez
  → Decide: preciso de uma tool? → Sim! → get_vaccination_status
  → Retorna: { tool_call: "get_vaccination_status", args: { petId: "abc123" } }
          |
          ▼
[API ROUTE - Next.js] (recebe o tool call)
  → Executa a função no Supabase:
      SELECT * FROM pet_vaccines WHERE pet_id = 'abc123'
  → Recebe: [{ vaccine: "V10", due_date: "2025-01-01", status: "overdue" }]
  → Devolve resultado para o modelo
          |
          ▼
[MODELO DE IA - Claude Sonnet] (continua)
  → Agora tem os dados reais
  → Gera a resposta final em streaming
          |
          ▼
[FRONTEND - React]
  → Recebe os tokens um a um via SSE (Server-Sent Events)
  → Exibe em tempo real enquanto chegam
  → Renderiza a resposta completa
```

Ponto-chave: A IA não tem acesso direto ao banco de dados. Ela **solicita** dados através de tools, e você (o servidor) **executa** e **devolve** os resultados. O modelo orquestra, mas você controla os dados.

---

3. Tokens — A Unidade de Trabalho da IA

3.1 O que é um token?

Token não é uma palavra. É um pedaço de texto que o modelo aprendeu a reconhecer como uma unidade.

```
"Bolinha está com parvovírus"
→ ["Bol", "inha", " está", " com", " par", "vov", "írus"]
   7 tokens (aprox.)

"cat" = 1 token
"cachorro" = 2 tokens ("cach", "orro")
"parvovírus" = 4 tokens (palavras raras = mais tokens)
```

Regra prática: 1 token ≈ 0.75 palavras em inglês, ≈ 0.6 palavras em português (pt-BR usa mais tokens).

3.2 Por que tokens importam?

```
Claude Sonnet 4.6:
  Preço entrada: ~$3 por 1 milhão de tokens
  Preço saída:   ~$15 por 1 milhão de tokens

Uma consulta no VetCopilot:
  System prompt:    ~800 tokens
  Contexto do pet:  ~500 tokens
  Histórico chat:   ~300 tokens
  Pergunta:         ~50 tokens
  ──────────────────────────
  Total entrada:    ~1.650 tokens = $0.005
  Resposta (saída): ~300 tokens = $0.0045
  ──────────────────────────
  Custo por consulta: ~$0.01 (1 centavo de dólar)
  100 consultas/dia = $1/dia = $30/mês
```

3.3 Otimização de tokens no AgendaVet

```
// RUIM — envia dados desnecessários
const petContext = JSON.stringify(pet) // inclui IDs internos, timestamps, etc.
// ~800 tokens
```

```
// BOM — envia apenas o que a IA precisa saber
const petContext = `
Pet: ${pet.name}, ${pet.species}, ${pet.breed}
Idade: ${calculateAge(pet.birth_date)}
Peso: ${pet.weight}kg
Tutor: ${pet.owner_name}
`
// ~50 tokens — 94% de economia
```

---

4. Context Window — A Memória de Curto Prazo

4.1 O conceito

O context window é o limite máximo de tokens que o modelo pode "ver" de uma vez. Tudo que está dentro dele é processado. O que ficou fora, o modelo não sabe que existe.

```
Claude Sonnet 4.6: 200.000 tokens de context window
= ~150.000 palavras em pt-BR
= um livro de 500 páginas inteiro

GPT-4o: 128.000 tokens
= ~95.000 palavras
```

4.2 Como o chat "lembra" de conversas anteriores

O modelo não tem memória. A cada chamada, você envia **todo o histórico**:

```
// Turno 1
messages = [
  { role: "system", content: "Você é VetCopilot..." },
  { role: "user",   content: "O Bolinha tem tosse" },
]

// Turno 2 — você adiciona a resposta anterior + nova mensagem
messages = [
  { role: "system",    content: "Você é VetCopilot..." },
```

```
  { role: "user",      content: "O Bolinha tem tosse" },
  { role: "assistant", content: "Pode ser traqueobronquite..." },
  { role: "user",      content: "Quais exames pedir?" },
]

// Turno 3 — acumula mais
messages = [
  { role: "system",    content: "Você é VetCopilot..." },
  { role: "user",      content: "O Bolinha tem tosse" },
  { role: "assistant", content: "Pode ser traqueobronquite..." },
  { role: "user",      content: "Quais exames pedir?" },
  { role: "assistant", content: "Radiografia torácica e..." },
  { role: "user",      content: "Quanto custa em média?" },
]
```

Problema: Consultas longas ficam caras e podem chegar no limite do context window.

Solução — Summarização:

```
// Quando o histórico passa de X tokens, resumir as mensagens antigas
async function summarizeHistory(messages: Message[]): Promise<string> {
  const response = await ai.generateText({
    model: "claude-haiku-4-5", // modelo barato para resumir
    prompt: `Resuma esta consulta veterinária em 3-4 frases:
    ${messages.map(m => `${m.role}: ${m.content}`).join('\n')}`,
  })
  return response.text
}

// Substitui N mensagens antigas por 1 resumo compacto
```

---

5. System Prompt — Como Programar o Comportamento da IA

5.1 O que é o System Prompt

É a "programação" do modelo para aquela sessão. Definido antes de qualquer mensagem do usuário. O modelo tratará as instruções do system prompt com mais autoridade.

5.2 Anatomia de um bom System Prompt para o VetCopilot

```
// lib/vet-copilot/system-prompt.ts

export function buildSystemPrompt(context: PetContext): string {
  return `
## IDENTIDADE
Você é o VetCopilot, assistente clínico especializado em medicina veterinária
integrado ao sistema AgendaVet. Você AUXILIA veterinários — nunca substitui.

## PACIENTE ATUAL
${formatPetContext(context)}

## REGRAS ABSOLUTAS (nunca viole)
1. SEMPRE apresente diagnósticos como "possíveis" ou "diferenciais", nunca definitivos
2. SEMPRE inclua: "Decisão final é do veterinário responsável"
3. NUNCA calcule doses sem confirmar peso atual do paciente
4. NUNCA sugira eutanásia sem o veterinário perguntar explicitamente
5. Se não tiver certeza, diga "Não tenho dados suficientes para afirmar"

## COMPORTAMENTO
- Seja direto e clínico — evite prolixidade
- Use terminologia veterinária adequada
- Organize respostas com bullets quando houver listas
- Para diagnósticos, use formato: (1) mais provável → (N) menos provável
- Sempre cite se a informação veio do prontuário ou de conhecimento geral

## FONTES DE CONHECIMENTO
Baseie-se em:
- Dados do prontuário fornecidos (prioridade máxima)
- Diretrizes WSAVA e AAHA
- Farmacologia veterinária (Plumb's Veterinary Drug Handbook)
- Indique nível de evidência: [PRONTUÁRIO], [DIRETRIZ], [LITERATURA]

## ESPECIALIDADE
${context.veterinarian.specialty
  ? `O veterinário é especialista em ${context.veterinarian.specialty}.
     Calibre o nível técnico das respostas adequadamente.`
  : 'Clínica geral — respostas em nível de clínico geral.'}
`.trim()
}
```

5.3 Técnicas Avançadas de Prompt Engineering

Chain of Thought (Raciocínio em Cadeia):

```
// Força o modelo a pensar antes de responder
const prompt = `
Antes de responder, raciocine internamente:
1. Quais são os diagnósticos diferenciais?
2. Qual evidência no prontuário apoia cada um?
3. O que seria mais grave e precisa de atenção imediata?
Depois, apresente sua conclusão.

Pergunta: ${userMessage}
`
```

Few-Shot Learning (Exemplos no Prompt):

```
// Mostrar exemplos do formato esperado
const prompt = `
Ao sugerir exames, use exatamente este formato:

EXEMPLO:
Exames Recomendados:
• Hemograma completo — [MOTIVO: avaliar resposta inflamatória] — Prioridade: ALTA
• Radiografia torácica — [MOTIVO: descartar pneumonia] — Prioridade: MÉDIA

Agora responda: ${userMessage}
`
```

Role Prompting (Definição de Papel):

```
// Diferentes papéis para diferentes contextos
const roles = {
  emergency:    "Você é especialista em emergências veterinárias. Priorize sempre o que s
  dermatology:  "Você é dermatologista veterinário. Explore sistematicamente todas causas
  nutrition:    "Você é nutricionista veterinário. Foque em condição corporal e dieta.",
}
```

6. Tool Calling (Function Calling) — Como a IA Acessa Dados Reais

6.1 O Conceito Fundamental

A IA não acessa banco de dados diretamente. O fluxo é:

```
IA detecta que precisa de dados
    ↓
IA retorna uma "intenção de chamada" (JSON)
    ↓
SEU CÓDIGO executa a função real
    ↓
SEU CÓDIGO devolve o resultado para a IA
    ↓
IA gera a resposta final com os dados reais
```

6.2 Como as Tools são definidas no AgendaVet

```ts
// lib/vet-copilot/tools.ts

import { tool } from 'ai'
import { z } from 'zod'

export const vetCopilotTools = {

  // TOOL 1: Buscar informações do pet
  get_pet_info: tool({
    description: `
      Busca dados básicos do paciente: nome, espécie, raça, idade, peso,
      cor, microchip, status reprodutivo e informações do tutor.
      Use quando precisar confirmar dados do paciente.
    `,
    parameters: z.object({
      petId: z.string().describe('ID único do pet no sistema')
    }),
    execute: async ({ petId }) => {
      const { data } = await supabase
        .from('pets')
        .select('*, profiles(full_name, phone)')
        .eq('id', petId)
        .single()
```

```
    return formatPetInfo(data) // retorna texto formatado, não JSON bruto
  }
}),

// TOOL 2: Calcular dosagem de medicamento
calculate_medication_dosage: tool({
  description: `
    Calcula a dose de um medicamento com base no peso do animal.
    Retorna dose, frequência, duração e alertas de segurança.
    SEMPRE use esta tool para cálculos — nunca calcule manualmente.
  `,
  parameters: z.object({
    medication: z.string().describe('Nome do medicamento (ex: Amoxicilina)'),
    petWeight:  z.number().describe('Peso do animal em kg'),
    species:    z.enum(['canino', 'felino', 'outros']),
    indication: z.string().describe('Para qual condição é o medicamento')
  }),
  execute: async ({ medication, petWeight, species, indication }) => {
    // Busca na tabela de doses ou chama API externa de farmacologia
    const dosage = await getDosageFromDatabase(medication, species)

    if (!dosage) {
      return `Medicamento "${medication}" não encontrado na base.
              Consulte Plumb's Veterinary Drug Handbook.`
    }

    const doseMin = dosage.mg_per_kg_min * petWeight
    const doseMax = dosage.mg_per_kg_max * petWeight

    return `
      Medicamento: ${medication}
      Dose calculada: ${doseMin}mg - ${doseMax}mg por dose
      Frequência: ${dosage.frequency}
      Via: ${dosage.route}
      Duração típica: ${dosage.duration}
      ⚠️ Alertas: ${dosage.warnings}
      [Cálculo baseado em: ${petWeight}kg × ${dosage.mg_per_kg_min}-${dosage.mg_per_kg
    `
  }
}),

// TOOL 3: Busca semântica (RAG) — Fase 2
search_clinical_knowledge: tool({
  description: `
    Busca em base de conhecimento veterinário (diretrizes WSAVA/AAHA,
    protocolos clínicos, bulas). Use quando precisar de informação
    baseada em evidência que não está no prontuário.
  `,
  parameters: z.object({
    query: z.string().describe('O que você quer buscar'),
    topic: z.enum([
      'vacinação', 'parasitologia', 'cirurgia',
      'farmacologia', 'nutrição', 'emergência', 'geral'
    ])
```

```
    }),
    execute: async ({ query, topic }) => {
      // Fase 2: Busca vetorial no Supabase pgvector
      const embedding = await generateEmbedding(query)
      const results = await searchVectorStore(embedding, topic, limit=3)
      return formatSearchResults(results)
    }
  }),
}
```

6.3 Como o Modelo Decide Usar uma Tool

O modelo lê a `description` da tool e decide se precisa usá-la. Por isso, **a descrição é crítica**:

```
// DESCRIÇÃO RUIM — vaga
description: "Busca dados do pet"

// DESCRIÇÃO BOA — específica sobre quando e por que usar
description: `
  Busca TODOS os dados clínicos do paciente: histórico de consultas,
  diagnósticos anteriores, medicações em uso, alergias conhecidas.
  Use quando o veterinário perguntar sobre histórico, condições existentes,
  ou quando precisar de contexto para um diagnóstico diferencial.
  NÃO use para dados básicos (nome, raça) — esses já estão no contexto.
`
```

6.4 Tool Calling Paralelo

O modelo pode chamar múltiplas tools ao mesmo tempo:

```
Pergunta: "Resuma o Bolinha e verifique as vacinas"
    ↓
IA chama simultaneamente:
  → get_pet_info(petId)        ← em paralelo
  → get_vaccination_status(petId)  ← em paralelo
    ↓
Ambos retornam → IA monta resposta completa
```

Isso é gerenciado automaticamente pelo Vercel AI SDK com `streamText`.

7. Streaming — Por que a IA "Digita" em Tempo Real

7.1 Como funciona

Sem streaming, você esperaria 10-15 segundos para receber a resposta inteira. Com streaming, os tokens chegam assim que são gerados (~50ms por token).

```
// app/api/vet-copilot/route.ts

export async function POST(req: Request) {
  const { messages, petId } = await req.json()

  // Monta contexto do pet
  const context = await buildPetContext(petId)

  // Chama o modelo com streaming
  const result = streamText({
    model: openrouter('anthropic/claude-sonnet-4-6'),
    system: buildSystemPrompt(context),
    messages,
    tools: vetCopilotTools,
    maxSteps: 5, // máximo de rounds de tool calling
    onStepFinish: async ({ toolResults }) => {
      // Log para auditoria após cada tool call
      await logToolUsage(toolResults)
    },
  })

  // Retorna stream para o frontend
  return result.toDataStreamResponse()
}
```

7.2 No Frontend — Consumindo o Stream

```
// components/vet-copilot/vet-copilot-chat.tsx

import { useChat } from 'ai/react'

export function VetCopilotChat({ petId }: { petId: string }) {
```

```
  const { messages, input, handleInputChange, handleSubmit, isLoading } = useChat({
    api: '/api/vet-copilot',
    body: { petId }, // enviado em cada request
    onFinish: (message) => {
      // Salva no Supabase quando a resposta completa chegar
      saveCopilotInteraction(message)
    }
  })

  return (
    <div>
      {messages.map(m => (
        <div key={m.id} className={m.role === 'user' ? 'text-right' : 'text-left'}>
          {/* Renderiza markdown das respostas */}
          <ReactMarkdown>{m.content}</ReactMarkdown>
        </div>
      ))}

      {/* Indicador de loading enquanto a IA "pensa" */}
      {isLoading && <TypingIndicator />}

      <form onSubmit={handleSubmit}>
        <input value={input} onChange={handleInputChange} />
        <button type="submit">Enviar</button>
      </form>
    </div>
  )
}
```

8. RAG — Retrieval Augmented Generation

8.1 O Problema que o RAG Resolve

Os modelos de IA têm uma data de corte de conhecimento. Eles não sabem:

- As últimas diretrizes da WSAVA (2024/2025)
- O protocolo específico da SUA clínica
- Bulas de medicamentos lançados recentemente
- Casos clínicos documentados por você ao longo dos anos

O RAG resolve isso: **antes de responder, busca os documentos relevantes e os inclui no contexto.**

8.2 Como RAG Funciona — Passo a Passo

```
FASE DE INGESTÃO (offline, feita uma vez):
  Documento PDF (diretriz WSAVA 2024)
        │
        ▼
  Divide em chunks (pedaços de ~500 tokens cada)
        │
        ▼
  Para cada chunk → gera um embedding (vetor de 1536 dimensões)
        │
        ▼
  Salva (texto + vetor) no Supabase pgvector

FASE DE QUERY (online, a cada pergunta):
  Usuário pergunta: "Protocolo de vacinação para gatos adultos?"
        │
        ▼
  Gera embedding da pergunta
        │
        ▼
  Busca os 3-5 chunks mais similares por distância vetorial
        │
        ▼
  Inclui esses chunks no prompt da IA
        │
        ▼
  IA responde usando o conteúdo dos chunks + seu conhecimento geral
```

8.3 Implementação Completa para o AgendaVet

Passo 1 — Schema no Supabase:

```sql
-- Habilitar extensão pgvector
CREATE EXTENSION vector;

-- Tabela de documentos veterinários
CREATE TABLE vet_knowledge_base (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  content     TEXT NOT NULL,          -- texto original do chunk
  embedding   VECTOR(1536),           -- vetor gerado pelo modelo
  source      TEXT,                   -- ex: "WSAVA 2024", "AAHA 2023"
  topic       TEXT,                   -- ex: "vacinação", "parasitologia"
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```sql
-- Índice para busca rápida por similaridade
CREATE INDEX ON vet_knowledge_base
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Função SQL para busca semântica
CREATE OR REPLACE FUNCTION search_vet_knowledge(
  query_embedding VECTOR(1536),
  match_topic     TEXT,
  match_count     INT DEFAULT 5
)
RETURNS TABLE(id UUID, title TEXT, content TEXT, source TEXT, similarity FLOAT)
LANGUAGE SQL AS $$
  SELECT id, title, content, source,
         1 - (embedding <=> query_embedding) AS similarity
  FROM   vet_knowledge_base
  WHERE  topic = match_topic OR match_topic = 'geral'
  ORDER  BY embedding <=> query_embedding
  LIMIT  match_count;
$$;
```

Passo 2 — Pipeline de Ingestão:

```typescript
// scripts/ingest-knowledge.ts
import { createClient } from '@supabase/supabase-js'

async function ingestDocument(pdfPath: string, source: string, topic: string) {
  // 1. Extrai texto do PDF
  const text = await extractTextFromPDF(pdfPath)

  // 2. Divide em chunks de ~500 tokens com overlap
  const chunks = splitIntoChunks(text, {
    chunkSize: 500,
    overlap: 50,  // sobreposição para não perder contexto nas bordas
  })

  console.log(`Processando ${chunks.length} chunks de "${source}"...`)

  // 3. Para cada chunk, gera embedding e salva
  for (const chunk of chunks) {
    const embedding = await generateEmbedding(chunk) // ver seção 9

    await supabase.from('vet_knowledge_base').insert({
      title:     `${source} — chunk ${chunks.indexOf(chunk) + 1}`,
      content:   chunk,
      embedding: embedding,
      source:    source,
      topic:     topic,
    })
```

```
    // Rate limiting — APIs de embedding têm limite de req/min
    await sleep(100)
  }

  console.log(`✅ "${source}" ingerido com sucesso!`)
}

// Uso:
await ingestDocument('./docs/wsava-vaccination-guidelines-2022.pdf', 'WSAVA 2022', 'vaci
await ingestDocument('./docs/aaha-pain-management-2022.pdf', 'AAHA Pain 2022', 'analgesi
await ingestDocument('./docs/plumbs-drug-handbook.pdf', 'Plumb\'s 9th Ed', 'farmacologia
```

Passo 3 — Busca Semântica na Tool:

```
// lib/vet-copilot/tools.ts

search_clinical_knowledge: tool({
  description: 'Busca em base de conhecimento veterinário baseada em evidência',
  parameters: z.object({
    query: z.string(),
    topic: z.enum(['vacinação', 'parasitologia', 'farmacologia', 'geral'])
  }),
  execute: async ({ query, topic }) => {
    // Gera embedding da query
    const queryEmbedding = await generateEmbedding(query)

    // Busca os chunks mais relevantes
    const { data: results } = await supabase.rpc('search_vet_knowledge', {
      query_embedding: queryEmbedding,
      match_topic:     topic,
      match_count:     4
    })

    if (!results || results.length === 0) {
      return 'Nenhuma informação encontrada na base de conhecimento.'
    }

    // Formata para a IA consumir
    return results.map(r => `
      FONTE: ${r.source} (similaridade: ${(r.similarity * 100).toFixed(0)}%)
      CONTEÚDO: ${r.content}
    `).join('\n---\n')
  }
})
```

9. Embeddings — Como a IA Entende Semântica

9.1 O Conceito

Um embedding é um vetor numérico que representa o **significado** de um texto. Textos com significado parecido ficam próximos no espaço vetorial.

```
"protocolo de vacinação para cães"  → [0.12, -0.34, 0.89, ...]  (1536 números)
"vacinas caninas e calendário"      → [0.11, -0.32, 0.91, ...]  (próximo!)
"receita de bolo de chocolate"      → [-0.78, 0.45, -0.12, ...] (distante!)
```

A distância entre vetores mede a similaridade semântica. Isso permite buscar por **significado**, não por palavras exatas.

9.2 Gerando Embeddings no Projeto

```ts
// lib/vet-copilot/embeddings.ts

import OpenAI from 'openai'

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY })

export async function generateEmbedding(text: string): Promise<number[]> {
  const response = await openai.embeddings.create({
    model: 'text-embedding-3-small', // barato: $0.02/1M tokens
    input: text.slice(0, 8000),      // limite do modelo
  })
  return response.data[0].embedding
}

// Alternativa com Gemini (sem custo adicional se já usa Gemini)
import { GoogleGenerativeAI } from '@google/generative-ai'

const gemini = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!)

export async function generateEmbeddingGemini(text: string): Promise<number[]> {
  const model = gemini.getGenerativeModel({ model: 'text-embedding-004' })
  const result = await model.embedContent(text)
  return result.embedding.values
}
```

9.3 Casos de Uso de Embeddings no AgendaVet

Além do RAG, embeddings têm outros usos poderosos:

```
1. BUSCA INTELIGENTE DE PRONTUÁRIOS
   Veterinário digita: "casos de insuficiência renal em gatos idosos"
   Sistema busca semanticamente entre todos os prontuários
   Resultado: casos similares mesmo com palavras diferentes

2. DETECÇÃO DE SIMILARIDADE ENTRE CASOS
   Novo caso com sintomas X, Y, Z
   Busca casos históricos similares
   Exibe: "3 casos similares encontrados — ver diagnósticos anteriores"

3. SUGESTÃO AUTOMÁTICA DE DIAGNÓSTICOS
   Com base nos sintomas atuais + embeddings de casos históricos
   Sugere os diagnósticos mais comuns para esse perfil
```

10. A Arquitetura do VetCopilot no AgendaVet

10.1 Visão Completa do Sistema Atual

```
┌──────────────────────────────────────────────────────────────────┐
│                    USUÁRIO (Veterinário)                         │
│              Browser / App Expo (tablet/celular)                 │
└──────────────────────────┬───────────────────────────────────────┘
                           │ HTTPS
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│              NEXT.JS APP (Vercel Edge)                           │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              /app/api/vet-copilot/route.ts               │   │
│  │                                                          │   │
│  │  1. Verifica JWT (Supabase Auth)                         │   │
│  │  2. Carrega contexto do pet (gatherPetContext)           │   │
│  │  3. Monta system prompt (buildSystemPrompt)              │   │
│  │  4. Chama streamText() com tools                         │   │
│  │  5. Executa tools quando solicitado                      │   │
│  │  6. Faz streaming da resposta                            │   │
│  └──────────────────────────────────────────────────────────┘   │
```

```
└──────────────────┴──────────────────────────────┴──────────────────┘
                   │                              │
                   │                              │
                   ▼ API CALL                     ▼ QUERIES
┌──────────────────────────┐    ┌──────────────────────────────┐
│   OPENROUTER (Proxy)     │    │   SUPABASE (PostgreSQL)      │
│                          │    │                              │
│  claude-sonnet-4-6       │    │  • pets                      │
│  claude-opus-4-6         │    │  • appointments              │
│  gpt-4o (fallback)       │    │  • pet_vaccines              │
│                          │    │  • pet_exams                 │
│  Vantagens:              │    │  • pet_observations          │
│  - Um billing só         │    │  • pet_prescriptions         │
│  - Fallback automático   │    │  • vet_knowledge_base (RAG)  │
│  - Monitoramento unif.   │    │  • copilot_interactions (log) │
└──────────────────────────┘    └──────────────────────────────┘
```

10.2 Fluxo de Dados Detalhado por Tipo de Pergunta

Caso A — Pergunta sobre dados do pet:

```
"O Bolinha tem alergia a algum medicamento?"
→ IA usa tool: get_pet_info(petId: "abc")
→ Supabase: SELECT allergies FROM pets WHERE id = 'abc'
→ Retorna: "Amoxicilina — reação alérgica (2023)"
→ IA responde: "⚠️ SIM — Bolinha tem alergia registrada a Amoxicilina..."
```

Caso B — Pergunta de conhecimento geral:

```
"Qual a dose de Prednisolona para gatos?"
→ IA usa tool: calculate_medication_dosage(med: "Prednisolona", weight: X, species: "felino")
→ Busca na tabela de doses
→ IA responde com dose calculada + alertas
```

Caso C — Pergunta clínica complexa (Fase 2 — RAG):

```
"Qual o protocolo WSAVA para vacinação de filhotes?"
→ IA usa tool: search_clinical_knowledge(query: "vacinação filhotes", topic: "vacinação")
→ Busca vetorial no Supabase pgvector
→ Recupera chunks relevantes das diretrizes WSAVA
→ IA responde citando a fonte: "[WSAVA 2022] Protocolo recomendado..."
```

11. Como Escrever Prompts Poderosos

11.1 Os 7 Elementos de um Prompt Eficaz

```
1. PAPEL   — quem a IA é neste contexto
2. OBJETIVO — o que ela deve fazer
3. CONTEXTO — informações relevantes para a tarefa
4. FORMATO  — como a resposta deve ser estruturada
5. TOM      — como a IA deve se comunicar
6. RESTRIÇÕES — o que ela NÃO deve fazer
7. EXEMPLOS — few-shot para calibrar o comportamento
```

11.2 Exemplos Comparativos

Prompt fraco:

```
"Analise este pet e diga se tem algum problema."
```

Resultado: resposta vaga, genérica, sem estrutura.

Prompt forte:

```
Você é um clínico veterinário sênior auxiliando durante uma consulta.

PACIENTE:
- Nome: ${pet.name}
- Espécie: ${pet.species} | Raça: ${pet.breed}
- Idade: ${pet.age} | Peso: ${pet.weight}kg
- Queixa principal: ${consultation.complaint}

HISTÓRICO RELEVANTE:
${formatRecentHistory(pet.history, limit=3)}

TAREFA:
1. Liste os 3 diagnósticos diferenciais mais prováveis (ordem decrescente de probabilidade)
2. Para cada diagnóstico, indique:
   - Evidências que suportam (baseadas no histórico acima)
   - Exames confirmatórios sugeridos
   - Urgência: [IMEDIATA / 24h / ELETIVA]
3. Se houver qualquer sinal de emergência, destaque em ⚠️
```

```
FORMATO DE RESPOSTA:
### Diagnósticos Diferenciais

**1. [Nome do diagnóstico]** — Probabilidade: Alta/Média/Baixa
- Evidências: ...
- Exames: ...
- Urgência: ...

RESTRIÇÕES:
- Não inclua diagnósticos sem base nos dados fornecidos
- Não use jargões desnecessários — o veterinário pode ser recém-formado
```

11.3 Prompt Engineering para Casos Específicos

Para resumo de prontuário:

```
export const SUMMARIZE_HISTORY_PROMPT = `
Você vai resumir o histórico médico de ${pet.name} para auxiliar o veterinário
durante a consulta de hoje.

HISTÓRICO (últimos 12 meses):
${medicalHistory}

CRIE UM RESUMO com:
1. CONDIÇÕES ATIVAS: doenças/condições em acompanhamento
2. MEDICAÇÕES EM USO: nome + dose + frequência
3. ALERGIAS: se houver
4. ÚLTIMAS CONSULTAS: data + motivo + resolução (máx 3)
5. PONTOS DE ATENÇÃO: qualquer coisa relevante para hoje

Seja conciso. Máximo 200 palavras. Use bullets.
`
```

Para cálculo de dose:

```
export const DOSAGE_CALC_PROMPT = `
Calcule a dose para o caso abaixo.

MEDICAMENTO: ${medication}
PACIENTE: ${pet.species}, ${pet.weight}kg, ${pet.age}
INDICAÇÃO: ${indication}

RESPONDA EXATAMENTE NESTE FORMATO:
---
DOSE: X mg/kg = Y mg por dose (para ${pet.weight}kg)
```

```
FREQUÊNCIA: a cada X horas / X vezes ao dia
VIA: oral / IM / IV / subcutânea
DURAÇÃO: X dias (típico para ${indication})
FORMA FARMACÊUTICA SUGERIDA: [comprimido/suspensão/injetável]

⚠️ ALERTAS:
- [qualquer contraindicação relevante]
- [ajuste necessário se doença renal/hepática]
---
FONTE: [Plumb's Veterinary Drug Handbook / WSAVA / outro]
`
```

---

12. Gerenciamento de Custos e Tokens

12.1 Estratégia de Modelos por Tarefa

A regra de ouro: **use o modelo mais barato que resolve o problema.**

```
// lib/vet-copilot/model-selector.ts

type TaskComplexity = 'simple' | 'moderate' | 'complex'

export function selectModel(task: TaskComplexity): string {
  switch (task) {
    case 'simple':
      // Resumos, formatação, perguntas diretas
      // Custo: ~$0.001 por chamada
      return 'anthropic/claude-haiku-4-5'

    case 'moderate':
      // Maioria das consultas clínicas
      // Custo: ~$0.01 por chamada
      return 'anthropic/claude-sonnet-4-6'

    case 'complex':
      // Diagnósticos raros, casos oncológicos, análise de exames complexos
      // Custo: ~$0.10 por chamada
      return 'anthropic/claude-opus-4-6'
  }
}

// Detector de complexidade baseado na query
```

```
export function detectComplexity(userMessage: string): TaskComplexity {
  const complexKeywords = [
    'oncologia', 'cardiopatia', 'neurologia', 'diagnóstico diferencial complexo',
    'interação medicamentosa', 'prognóstico', 'protocolo quimioterapia'
  ]

  const simpleKeywords = [
    'dose', 'peso', 'vacina', 'vermífugo', 'resumo', 'data', 'quando'
  ]

  if (complexKeywords.some(kw => userMessage.toLowerCase().includes(kw))) {
    return 'complex'
  }
  if (simpleKeywords.some(kw => userMessage.toLowerCase().includes(kw))) {
    return 'simple'
  }
  return 'moderate'
}
```

12.2 Cache de Respostas

```
// lib/vet-copilot/cache.ts

// Algumas perguntas são idênticas entre veterinários
// Ex: "dose de Amoxicilina para cão de 10kg" — mesma resposta sempre

export async function getCachedResponse(cacheKey: string): Promise<string | null> {
  const { data } = await supabase
    .from('ai_response_cache')
    .select('response')
    .eq('cache_key', cacheKey)
    .gt('expires_at', new Date().toISOString())
    .single()

  return data?.response ?? null
}

export async function setCachedResponse(
  cacheKey: string,
  response: string,
  ttlHours = 24
) {
  const expiresAt = new Date()
  expiresAt.setHours(expiresAt.getHours() + ttlHours)

  await supabase.from('ai_response_cache').upsert({
    cache_key:  cacheKey,
    response:   response,
    expires_at: expiresAt.toISOString(),
```

```
  })
}

// Uso na API route:
const cacheKey = `dosage:${medication}:${species}:${Math.round(weight)}`
const cached = await getCachedResponse(cacheKey)
if (cached) return cached // economiza 100% do custo do LLM
```

12.3 Monitoramento de Gastos em Tempo Real

```
// lib/analytics/cost-tracker.ts

interface AIUsageLog {
  id:              string
  model:           string
  input_tokens:    number
  output_tokens:   number
  cost_usd:        number
  veterinarian_id: string
  pet_id:          string
  task_type:       string
  created_at:      Date
}

const MODEL_COSTS = {
  'claude-haiku-4-5':    { input: 0.25,  output: 1.25  }, // $ por 1M tokens
  'claude-sonnet-4-6':   { input: 3.00,  output: 15.00 },
  'claude-opus-4-6':     { input: 15.00, output: 75.00 },
}

export async function logAIUsage(params: {
  model: string
  inputTokens: number
  outputTokens: number
  veterinarianId: string
  petId: string
  taskType: string
}) {
  const costs = MODEL_COSTS[params.model]
  const costUsd =
    (params.inputTokens  / 1_000_000 * costs.input) +
    (params.outputTokens / 1_000_000 * costs.output)

  await supabase.from('ai_usage_logs').insert({
    model:           params.model,
    input_tokens:    params.inputTokens,
    output_tokens:   params.outputTokens,
    cost_usd:        costUsd,
    veterinarian_id: params.veterinarianId,
```

```
    pet_id:          params.petId,
    task_type:       params.taskType,
  })
}

// Dashboard de custos:
// SELECT DATE(created_at), SUM(cost_usd), COUNT(*)
// FROM ai_usage_logs
// GROUP BY DATE(created_at)
// ORDER BY 1 DESC
```

13. O que Pode Ser Implementado — Mapa Completo

Fase Atual (MVP — Implementado)

```
✅ VetCopilot com tool calling
✅ 7 tools de acesso a dados do Supabase
✅ Streaming de respostas
✅ Interface de chat
✅ System prompt especializado
✅ Disclaimer de segurança
```

Fase 2 — RAG (Próxima — Alta Prioridade)

```
○ Supabase pgvector configurado
○ Pipeline de ingestão de documentos PDF
○ WSAVA/AAHA guidelines indexadas
○ Tool search_clinical_knowledge com busca real
○ Respostas citando fontes com nível de evidência
```

Fase 3 — Análise Diagnóstica

```
○ Matriz de diagnósticos diferenciais por sintoma/espécie
○ Interpretação automática de hemograma
```

```
  - Valores de referência por espécie/idade
  - Alertas de valores críticos
  - Sugestão de diagnóstico baseada em alterações
○ Interpretação de bioquímica
○ Sistema de scoring de gravidade (IRIS para doença renal, etc.)
```

Fase 4 — Geração de Documentos

```
○ Geração de anamnese estruturada em SOAP
   S (Subjective): queixas do tutor
   O (Objective):  exame físico
   A (Assessment): diagnóstico/hipóteses
   P (Plan):       plano terapêutico
○ Receituário com instruções automáticas
○ Relatório de alta para o tutor (linguagem acessível)
○ Sumário de consulta para prontuário
```

Fase 5 — Visão Computacional

```
○ Análise de radiografias (Claude Vision / GPT-4 Vision)
○ Análise de fotos de lesões cutâneas
○ Comparação temporal (evolução de lesões)
○ Medições automatizadas em imagens
```

Fase 6 — Agente Proativo

```
○ Alertas de interações medicamentosas em tempo real
○ Notificação de vacinas vencendo (3 dias antes)
○ Detecção de padrões: "Gatos desta raça com estes sintomas têm
  alto risco de doença cardíaca"
○ Análise de tendências populacionais da clínica
```

Fase 7 — App Tutor com IA

```
○ Chat simplificado para tutores (linguagem acessível)
○ Triagem automática de urgência
```

```
  ○ Orientações pós-consulta personalizadas
  ○ Lembretes inteligentes baseados no histórico do pet
```

Fase 8 — Fine-tuning (Longo Prazo)

```
  ○ Treinar modelo específico com dados da clínica
  ○ Aprender padrões locais (raças prevalentes na região, etc.)
  ○ Personalização do estilo do veterinário específico
```

---

14. Próximos Passos Concretos com Código

Passo 1 — Configurar pgvector no Supabase (1-2 horas)

```sql
-- Execute no SQL Editor do Supabase

-- 1. Habilitar extensão
CREATE EXTENSION IF NOT EXISTS vector;

-- 2. Criar tabela de base de conhecimento
CREATE TABLE IF NOT EXISTS vet_knowledge_base (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title      TEXT NOT NULL,
  content    TEXT NOT NULL,
  embedding  VECTOR(1536),
  source     TEXT NOT NULL,
  topic      TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Criar índice vetorial
CREATE INDEX IF NOT EXISTS vet_knowledge_embedding_idx
  ON vet_knowledge_base
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 50);

-- 4. Criar função de busca
```

```sql
CREATE OR REPLACE FUNCTION match_vet_knowledge(
  query_embedding VECTOR(1536),
  match_count     INT DEFAULT 4,
  filter_topic    TEXT DEFAULT NULL
)
RETURNS TABLE(id UUID, title TEXT, content TEXT, source TEXT, similarity FLOAT)
LANGUAGE PLPGSQL AS $$
BEGIN
  RETURN QUERY
  SELECT
    vkb.id,
    vkb.title,
    vkb.content,
    vkb.source,
    1 - (vkb.embedding <=> query_embedding) AS similarity
  FROM vet_knowledge_base vkb
  WHERE
    filter_topic IS NULL OR vkb.topic = filter_topic
  ORDER BY vkb.embedding <=> query_embedding
  LIMIT match_count;
END;
$$;
```

Passo 2 — Script de Ingestão de Documentos (3-4 horas)

```typescript
// scripts/ingest-vet-knowledge.ts
// Rode com: npx ts-node scripts/ingest-vet-knowledge.ts

import { createClient } from '@supabase/supabase-js'
import OpenAI from 'openai'
import fs from 'fs'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY! // chave service role para ingestão
)

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY })

function splitIntoChunks(text: string, chunkSize = 500, overlap = 50): string[] {
  const words  = text.split(' ')
  const chunks: string[] = []
  let start    = 0

  while (start < words.length) {
    const end   = Math.min(start + chunkSize, words.length)
    const chunk = words.slice(start, end).join(' ')
```

```
    chunks.push(chunk)
    start += chunkSize - overlap
  }

  return chunks.filter(c => c.trim().length > 100) // ignora chunks muito curtos
}

async function generateEmbedding(text: string): Promise<number[]> {
  const response = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: text,
  })
  return response.data[0].embedding
}

async function ingestText(text: string, source: string, topic: string) {
  const chunks = splitIntoChunks(text)
  console.log(`\n📚 Ingerindo "${source}" — ${chunks.length} chunks`)

  let processed = 0
  for (const chunk of chunks) {
    try {
      const embedding = await generateEmbedding(chunk)

      await supabase.from('vet_knowledge_base').insert({
        title:     `${source} — parte ${processed + 1}`,
        content:   chunk,
        embedding: embedding,
        source:    source,
        topic:     topic,
      })

      processed++
      if (processed % 10 === 0) {
        console.log(`  ✓ ${processed}/${chunks.length} chunks processados`)
      }

      await new Promise(r => setTimeout(r, 200)) // evita rate limit
    } catch (error) {
      console.error(`  ✗ Erro no chunk ${processed}:`, error)
    }
  }

  console.log(`✅ "${source}" concluído — ${processed} chunks`)
}

// EXEMPLO — ingerir texto direto (você pode adaptar para PDF depois)
const wsavaVaccinationGuide = `
# WSAVA Vaccination Guidelines 2022
## Princípios Fundamentais

Os programas de vacinação devem ser individualizados para cada paciente,
levando em conta fatores como estilo de vida, saúde geral, risco de exposição
e histórico vacinal prévio.
```

```
### Vacinas Core para Cães
Vacinas core são aquelas recomendadas para todos os cães independente do
estilo de vida:
- Raiva (obrigatória por lei)
- Cinomose (CDV)
- Hepatite Infecciosa Canina (CAV-2)
- Parvovirose (CPV-2)

Protocolo para filhotes:
- 6-8 semanas: CDV + CAV-2 + CPV-2
- 10-12 semanas: CDV + CAV-2 + CPV-2 (reforço)
- 14-16 semanas: CDV + CAV-2 + CPV-2 (reforço final)
- 12-16 semanas: Raiva (1ª dose)
- 12 meses: Reforço anual (avaliar título sorológico)
- Adultos: Reforço a cada 1-3 anos (baseado em sorologia)
[... continua com mais conteúdo ...]
`

await ingestText(wsavaVaccinationGuide, 'WSAVA Vaccination Guidelines 2022', 'vacinação'
```

Passo 3 — Atualizar a Tool de Busca (30 min)

```
// lib/vet-copilot/tools.ts — substituir a implementação mock

search_clinical_knowledge: tool({
  description: `
    Busca em base de conhecimento veterinário baseada em evidência.
    Contém: diretrizes WSAVA 2022, AAHA 2023, Plumb's Drug Handbook.
    Use quando precisar de: protocolos de vacinação, dosagens detalhadas,
    diretrizes terapêuticas, protocolos de emergência, base científica.
    NÃO use para dados do paciente específico — use get_pet_info para isso.
  `,
  parameters: z.object({
    query: z.string().describe('O que você quer buscar — seja específico'),
    topic: z.enum(['vacinação', 'parasitologia', 'farmacologia', 'cirurgia', 'geral'])
      .describe('Tema para filtrar a busca')
  }),
  execute: async ({ query, topic }) => {
    // Gera embedding da query
    const queryEmbedding = await generateEmbedding(query)

    // Busca no Supabase
    const { data, error } = await supabase.rpc('match_vet_knowledge', {
      query_embedding: queryEmbedding,
      match_count:     4,
      filter_topic:    topic === 'geral' ? null : topic,
    })

    if (error) {
```

```
      console.error('Erro na busca RAG:', error)
      return 'Erro ao buscar na base de conhecimento.'
    }

    if (!data || data.length === 0) {
      return 'Nenhuma informação específica encontrada. Baseie-se em conhecimento geral.
    }

    // Formata resultado para a IA
    const formatted = data
      .filter(r => r.similarity > 0.7) // filtra resultados pouco relevantes
      .map((r, i) => `
[Fonte ${i+1}: ${r.source}] (relevância: ${(r.similarity * 100).toFixed(0)}%)
${r.content}
      `.trim())
      .join('\n\n---\n\n')

    return formatted || 'Informações encontradas mas com baixa relevância para a query.'
  }
})
```

Passo 4 — Logging de Auditoria Completo (2 horas)

```
// app/api/vet-copilot/route.ts — versão com auditoria

export async function POST(req: Request) {
  const { messages, petId } = await req.json()

  // Autentica
  const { data: { user } } = await supabase.auth.getUser(
    req.headers.get('Authorization')?.replace('Bearer ', '') ?? ''
  )
  if (!user) return new Response('Unauthorized', { status: 401 })

  // Monta contexto
  const context = await gatherPetContext(petId)
  const startTime = Date.now()

  const result = streamText({
    model: openrouter(selectModel(detectComplexity(
      messages[messages.length - 1]?.content ?? ''
    ))),
    system:   buildSystemPrompt(context),
    messages,
    tools:    vetCopilotTools,
    maxSteps: 5,

    onFinish: async ({ usage, text, toolCalls }) => {
      const duration = Date.now() - startTime
```

```
      // Salva interação completa para auditoria
      await supabase.from('copilot_interactions').insert({
        veterinarian_id: user.id,
        pet_id:          petId,
        user_message:    messages[messages.length - 1]?.content,
        ai_response:     text,
        tools_used:      toolCalls?.map(t => t.toolName) ?? [],
        model_used:      result.model,
        input_tokens:    usage.promptTokens,
        output_tokens:   usage.completionTokens,
        cost_usd:        calculateCost(result.model, usage),
        latency_ms:      duration,
        created_at:      new Date().toISOString(),
      })
    }
  })

  return result.toDataStreamResponse()
}
```

15. Riscos, Limitações e Ética

15.1 Limitações Técnicas

LIMITAÇÃO	IMPACTO	MITIGAÇÃO
Alucinação	IA pode inventar doses ou diagnósticos	Sempre verificar com tools reais; never trust output cego
Data de corte	Modelo não conhece literatura após 2024	RAG com documentos atualizados
Context window	Históricos longos são truncados	Summarização automática
Latência	2-8 segundos por resposta	Streaming + feedback visual
Custo	$0.01-0.10 por consulta	Cache + seleção inteligente de modelo

15.2 Riscos Clínicos

```
RISCO ALTO:
  ❌ Calcular doses sem confirmar peso atualizado do paciente
  ❌ Sugerir diagnóstico definitivo baseado em sintomas vagos
  ❌ Recomendar medicamento sem verificar alergias conhecidas

RISCO MÉDIO:
  ⚠️ Resposta sem citar a fonte da informação
  ⚠️ Protocolo desatualizado (sem RAG com docs recentes)
  ⚠️ Sugestão sem considerar interações medicamentosas

MITIGAÇÕES IMPLEMENTADAS:
  ✅ SAFETY_PREFIX no system prompt
  ✅ Disclaimer obrigatório em todas respostas
  ✅ Tools com validação antes de calcular doses
  ✅ Log de auditoria de todas as interações
```

15.3 LGPD e Privacidade

```
// DADOS QUE NÃO DEVEM IR PARA A IA:
const sensitiveFields = [
  'cpf',        // CPF do tutor
  'rg',         // RG
  'address',    // endereço completo
  'phone',      // telefone (pode ser omitido)
  'payment_*',  // qualquer dado de pagamento
]

// Use apenas o necessário clinicamente:
const clinicalContext = {
  petName:   pet.name,       // OK — necessário
  species:   pet.species,    // OK — necessário clinicamente
  breed:     pet.breed,      // OK — necessário clinicamente
  age:       calculateAge(pet.birth_date), // OK — calculado
  weight:    pet.weight,     // OK — necessário para doses
  // ownerCpf: pet.owner_cpf  ← NÃO INCLUA
  // ownerAddress: ...        ← NÃO INCLUA
}
```

16. Glossário Técnico

TERMO	O QUE É
LLM	Large Language Model — modelo de linguagem de grande escala (Claude, GPT)
Token	Unidade de texto processada pelo modelo (~0.75 palavras em inglês)
Context Window	Limite máximo de tokens que o modelo processa de uma vez
System Prompt	Instruções dadas ao modelo antes da conversa começar
Tool Calling	Capacidade do modelo de solicitar execução de funções externas
Streaming	Envio da resposta token por token em vez de esperar o texto completo
RAG	Retrieval Augmented Generation — combinar busca em documentos + geração de texto
Embedding	Representação numérica (vetor) do significado semântico de um texto
pgvector	Extensão do PostgreSQL para armazenar e buscar vetores (embeddings)
Fine-tuning	Treinar um modelo base em dados específicos para especialização
RLHF	Reinforcement Learning from Human Feedback — como modelos aprendem preferências humanas
Transformer	Arquitetura neural que fundamenta todos os LLMs modernos
Atenção (Attention)	Mecanismo que permite o modelo focar nas partes relevantes do contexto
Temperature	Controla a criatividade/aleatoriedade (0 = determinístico, 1 = criativo)
Top-P / Top-K	Outros parâmetros de aleatoriedade das respostas
Alucinação	Quando o modelo gera informações plausíveis mas incorretas

Chain of Thought	Técnica de forçar o modelo a "pensar em voz alta" antes de responder
Few-Shot	Incluir exemplos no prompt para calibrar o comportamento
Zero-Shot	Pedir ao modelo que faça algo sem dar exemplos
Chunking	Dividir documentos longos em pedaços menores para indexação
Vector Store	Banco de dados otimizado para armazenar e buscar embeddings
Similarity Search	Busca por similaridade semântica (não por palavras exatas)
OpenRouter	Proxy que unifica acesso a múltiplos modelos de IA em uma API
AI SDK (Vercel)	Biblioteca TypeScript que simplifica integração com modelos de IA

Evolução Esperada do VetCopilot

```
HOJE (MVP)                    FASE 2 (RAG)                FASE 3+ (Autônomo)
──────────────────────        ──────────────────────      ──────────────────
Responde perguntas       →    Cita fontes científicas →   Sugere proativamente
Acessa prontuário             Busca diretrizes              Alerta interações
Calcula doses                 Base de conhecimento          Prediz riscos
Streaming                     Evidência baseada             Gera documentos
                              em dados reais                Análisa imagens
```

Última atualização: Março 2026 Baseado na arquitetura real do AgendaVet — AgendaVetVet e AgendaVetWeb